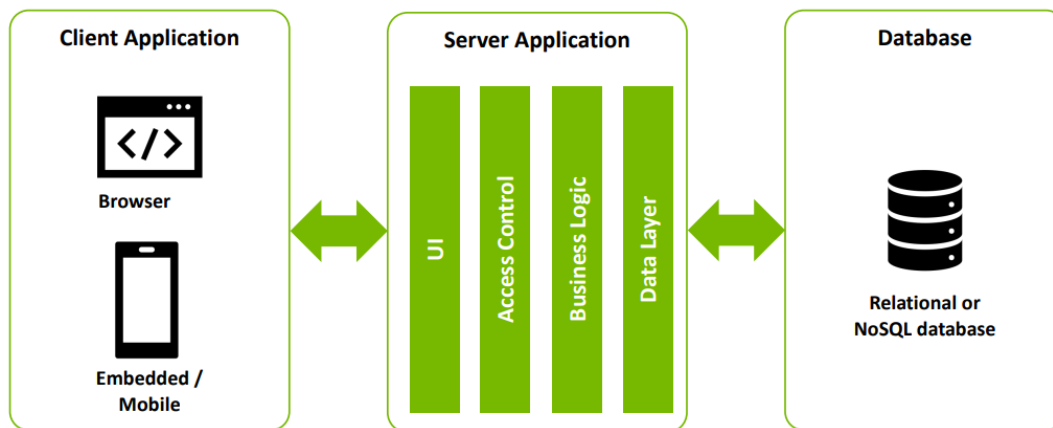


1. Aus welchen Komponenten besteht die Three-Tier Architecture?

Die **Three-Tier Architecture** besteht aus der **Presentation Tier**, der **Application Tier** und der **Data Tier**. Die **Presentation Tier** stellt die **Benutzeroberfläche (UI)** bereit und ermöglicht die **Interaktion mit dem Nutzer**, beispielsweise über einen **Webbrowser** oder eine **mobile App**. Die **Application Tier** enthält die **Geschäftslogik**, verarbeitet **Anfragen** und steuert den **Zugriff auf die Daten**. Die **Data Tier** ist für die **persistente Speicherung und Verwaltung der Daten** zuständig und besteht typischerweise aus **relationalen oder NoSQL-Datenbanken**. Durch die **klare Trennung der Schichten** wird die Anwendung **wartbarer, skalierbarer und flexibler**.



2. Was versteht man unter der Latenz und wo tritt sie auf?

Unter **Latenz** versteht man die **zeitliche Verzögerung** zwischen dem **Absenden einer Anfrage** und dem **Eintreffen der zugehörigen Antwort** in einem **verteilten System**. **Latenz tritt überall dort auf, wo Kommunikation über ein Netzwerk erfolgt**, insbesondere zwischen **Client und Server**, zwischen **mehreren Servern** sowie beim **Zugriff des Application Servers auf die Datenbank**. Sie entsteht durch **Netzwerkübertragung**, **Verarbeitung auf den beteiligten Systemen** und **physikalische Distanzen** und ist ein **zentrales Merkmal verteilter Anwendungen**.

3. Welche Vorteile besitzt eine verteilte Anwendung gegenüber einer Desktop-Anwendung?

Verteilte Anwendungen bieten gegenüber **Desktop-Anwendungen** insbesondere Vorteile in Bezug auf **Skalierbarkeit**, **Fehlertoleranz** und **Wartbarkeit**. Durch die **funktionale Trennung der Systemkomponenten** können einzelne Teile **unabhängig skaliert, aktualisiert oder ausgetauscht** werden. Zudem ermöglichen verteilte Anwendungen einen **orts- und geräteunabhängigen Zugriff**, eine **zentrale Datenhaltung** sowie eine höhere **Ausfallsicherheit**, da der **Ausfall einzelner Komponenten** nicht zwangsläufig zum **Stillstand des gesamten Systems** führt.

Functional Separation
• Systems are separated based on the functionality/services provided, capability and purpose of each entity
Inherent distribution
• Information, people, functionalities, and systems are inherently distributed
Resiliency
• The ability of a system to self-heal after damage, failure, load, or attack
Scalability
• The property of a system to handle a growing amount of work by adding resources to the system
Fault Tolerance
• The ability of a system to continue operating properly in the event of failure within some of its components

1. Was ist der Unterschied zwischen einem Application Server und einem Web Server?

Ein **Web Server** dient primär der **Auslieferung statischer Inhalte** wie **HTML**, **CSS** oder **JavaScript** und verarbeitet einfache **HTTP-Anfragen**, ohne komplexe Geschäftslogik auszuführen. Ein **Application Server** hingegen führt **Anwendungs- und Geschäftslogik** aus, verwaltet **Transaktionen**, **Sicherheitsmechanismen** und **Middleware-Dienste** und stellt dynamische Inhalte bereit. Während ein Web Server lediglich Inhalte bereitstellt, ist der Application Server für die **Verarbeitung der Anwendung** verantwortlich.

2. Hat ein Docker Container ein eigenes Betriebssystem?

Ein **Docker Container besitzt kein eigenes Betriebssystem**. Stattdessen **teilt er sich den Kernel des Host-Betriebssystems** und kapselt lediglich die **Anwendung**, deren **Abhängigkeiten** und **Bibliotheken**. Dadurch entsteht deutlich **weniger Overhead** als bei virtuellen Maschinen, da kein vollständiges Gastbetriebssystem benötigt wird.

3. Welches Betriebssystem muss installiert sein, um eine Container Engine auszuführen?

Um eine **Container Engine** wie **Docker** auszuführen, muss ein **Host-Betriebssystem mit Kernel-Unterstützung für OS-Virtualisierung** installiert sein, typischerweise ein **Linux-Betriebssystem**. Funktionen wie **Namespaces** und **Control Groups (cgroups)** sind notwendig, um **Isolation** und **Ressourcenverwaltung** der Container zu ermöglichen. Unter Windows oder macOS wird Docker daher meist über eine **virtuelle Linux-Umgebung** betrieben.

4. Worin unterscheiden sich virtuelle Maschinen und Docker Container?

Virtuelle Maschinen virtualisieren die **gesamte Hardware** und enthalten jeweils ein **eigenes Gastbetriebssystem**, das über einen **Hypervisor** ausgeführt wird, was zu **höherem Ressourcenverbrauch** führt. **Docker Container** hingegen nutzen **OS-Virtualisierung**, teilen sich den **Kernel des Host-Betriebssystems** und enthalten kein eigenes Betriebssystem. Dadurch sind Container **leichter, schneller startbar** und **ressourcenschonender**, während virtuelle Maschinen eine **stärkere Isolation** bieten.

1. Was ist der Unterschied zwischen einem Dockerfile und einem Docker Compose File?

Ein **Dockerfile** beschreibt, **wie ein Docker Image gebaut wird**, indem es die **Basis-Images, Dateikopien, Installationsschritte** und **Konfigurationen** einer einzelnen Anwendung definiert. Ein **Docker Compose File** hingegen beschreibt, **wie mehrere Container gemeinsam gestartet und konfiguriert werden**, inklusive **Services, Port-Mappings, Umgebungsvariablen** und **Abhängigkeiten**. Während das Dockerfile den **Build-Prozess eines Images** festlegt, dient Docker Compose der **Orchestrierung mehrerer Container**.

2. Welche Techniken bzw. Vorgehensweisen können angewendet werden, um Fehlkonfigurationen bei unerwartetem Verhalten eines laufenden Docker Containers zu identifizieren?

Zur Analyse von **Fehlkonfigurationen** können verschiedene **Docker-Debugging-Techniken** eingesetzt werden, insbesondere das **Auslesen der Container-Logs** mit `docker logs`, das **Untersuchen laufender Prozesse** mit `docker top` sowie das **Betreten des Containers** mittels `docker exec`, um Konfigurationen und Dateien direkt zu prüfen. Zusätzlich liefern Befehle wie `docker info` Informationen über die **Docker-Umgebung**, wodurch Konfigurations- oder Ressourcenprobleme identifiziert werden können.

3. Welches Protokoll nutzt ein Web Server, um statischen Content über das Netzwerk zur Verfügung zu stellen?

Ein **Web Server** nutzt das **Hypertext Transfer Protocol (HTTP)**, um **statischen Content** wie **HTML, CSS** und **JavaScript** über ein **Netzwerk oder das Internet** bereitzustellen. HTTP ermöglicht dabei die **standardisierte Kommunikation** zwischen **Client und Web Server** und ist die Grundlage für die Auslieferung statischer Webinhalte.

1. Was ist der Unterschied zwischen HTTP und HTTPS?

HTTP ist ein **unverschlüsseltes, textbasiertes Client-Server-Protokoll**, bei dem Daten im Klartext übertragen werden. **HTTPS** erweitert HTTP um eine **Transportverschlüsselung mittels TLS**, wodurch **Vertraulichkeit, Integrität und Authentizität** der Kommunikation sichergestellt werden. HTTPS schützt somit vor **Abhören** und **Manipulation** der übertragenen Daten.

2. Was beschreibt der Content-Type? (Kennen Sie ein Beispiel?)

Der **Content-Type** beschreibt den **Medientyp (MIME Type)** des Inhalts einer HTTP-Nachricht und gibt an, **wie der Body interpretiert werden soll**. Er wird im **HTTP Header** übertragen und ist zentral für die **korrekte Verarbeitung durch den Client**.

Ein Beispiel ist **Content-Type: text/html** für HTML-Seiten oder **Content-Type: application/json** für REST-APIs.

3. Welche wesentlichen Methoden existieren beim HTTP-Protokoll?

Zu den **wesentlichen HTTP-Methoden** gehören **GET** (Daten abrufen), **POST** (Daten senden bzw. Zustand ändern), **PUT** (Ressource vollständig ersetzen), **PATCH** (Ressource teilweise ändern) und **DELETE** (Ressource löschen). Ergänzend existieren Methoden wie **HEAD**, **OPTIONS** und **TRACE**, die vor allem für **Metainformationen** und **Kommunikationssteuerung** genutzt werden.

**4. Welche zwei Arten der Content Negotiation gibt es?

Ø Negotiation auch beim API?**

Es gibt **Server-driven Content Negotiation**, bei der der **Server anhand von HTTP-Headern** wie **Accept**, **Accept-Language** oder **Accept-Encoding** die passende Repräsentation auswählt, sowie **Agent-driven Content Negotiation**, bei der der **Client aktiv verschiedene Formate anfragt** und auf Serverantworten reagiert.

Content Negotiation wird auch bei APIs eingesetzt, insbesondere bei **REST-APIs**, um unterschiedliche **Datenformate** wie **JSON** oder **XML** bereitzustellen.

1. Welche HTTP-Parameter können verwendet werden, um Informationen bei einem GET-Request an den Server zu übertragen?

Bei einem **GET-Request** werden Informationen über **Query-Parameter** in der **URL** übertragen. Diese stehen nach einem **Fragezeichen (?)** und bestehen aus **Name-Wert-Paaren**, die mit **&** getrennt sind.

Beispiel:

GET /users?name=alex&limit=10

Query-Parameter werden häufig für **Filterung**, **Sortierung** und **Paging** verwendet.

2. Welche HTTP-Methoden dürfen den Zustand auf dem Server ändern?

Den **Zustand auf dem Server ändern** dürfen die HTTP-Methoden **POST**, **PUT**, **PATCH** und **DELETE**. **POST** erstellt neue Ressourcen, **PUT** ersetzt eine Ressource vollständig, **PATCH** ändert Teile einer Ressource und **DELETE** entfernt eine Ressource.

Die Methode **GET** darf den Serverzustand **nicht verändern**.

3. Wie können RESTful APIs spezifiziert bzw. dokumentiert werden?

RESTful APIs können mit dem **OpenAPI-Standard** (früher **Swagger**) **maschinell lesbar spezifiziert und dokumentiert** werden. OpenAPI beschreibt unter anderem **Endpunkte**, **HTTP-Methoden**, **Request- und Response-Strukturen**, **Statuscodes** und **Datenmodelle**.

In Java-Umgebungen kann dies automatisiert über **MicroProfile OpenAPI** erfolgen, wodurch die Dokumentation **stets konsistent zur Implementierung** bleibt.

4. Wie kann der Client den Server über Änderungen informieren?

Der Client informiert den Server über **Änderungen**, indem er **HTTP-Requests mit zustandsverändernden Methoden** wie **POST**, **PUT**, **PATCH** oder **DELETE** sendet. Die **Änderungsdaten** werden dabei im **Request Body** übertragen, typischerweise im Format **application/json**. Der Server verarbeitet die Anfrage und bestätigt die Änderung über einen **HTTP-Statuscode**.

1. Warum können Webhook-Callbacks normalerweise nur von einem Server verwendet werden?

Webhook-Callbacks können normalerweise nur von einem **Server** verwendet werden, da sie eine **öffentlich erreichbare HTTP-Endpoint-URL** erfordern, an die **HTTP-POST-Requests** gesendet werden. Browser oder Clients befinden sich häufig **hinter NAT oder Firewalls** und können daher **keine stabil erreichbaren Endpunkte** bereitstellen. Zudem erfordern Webhooks keine **persistente Verbindung**, sondern setzen auf **serverseitige Erreichbarkeit**.

2. Was ist der Unterschied zwischen Server Sent Events und WebSockets?

Server Sent Events (SSE) ermöglichen eine **unidirektionale Kommunikation vom Server zum Client** über eine **lang bestehende HTTP-Verbindung**, während **WebSockets** eine **bidirektionale, voll-duplex Kommunikation** zwischen Client und Server erlauben. SSE ist **textbasiert** und eignet sich für **Live-Updates**, während WebSockets sowohl **Text- als auch Binärdaten** unterstützen und für **interaktive Echtzeitanwendungen** verwendet werden.

3. Welche Voraussetzung brauchen SSE und WebSockets, die ein Webhook nicht braucht?

SSE und WebSockets benötigen eine **persistente, lang laufende Netzwerkverbindung** zwischen Client und Server. Diese Verbindung muss vom **Client aktiv aufgebaut und offen gehalten** werden. **Webhooks** hingegen benötigen **keine dauerhafte Verbindung**, da sie auf **einzelnen HTTP-Requests** basieren, die ereignisgesteuert ausgelöst werden.

4. Was unterscheidet eine Push- von einer Pullarchitektur?

In einer **Pull-Architektur** fragt der **Client regelmäßig aktiv beim Server nach neuen Daten** (Polling), unabhängig davon, ob sich der Zustand geändert hat. In einer **Push-Architektur** informiert der **Server den Client automatisch**, sobald **neue Informationen vorliegen**, wodurch unnötige Anfragen vermieden und **Echtzeitkommunikation** ermöglicht wird.

Klausur-Merksatz

Webhooks benötigen erreichbare Server-Endpunkte, **SSE** ist unidirektional, **WebSockets** bidirektional, beide brauchen persistente Verbindungen, und **Push-Architekturen** senden Updates ereignisgesteuert statt per **Polling**.

1. Was versteht man unter Head-of-Line-Blocking?

Head-of-Line-Blocking bezeichnet das Problem, dass bei **HTTP/1.1** mehrere **HTTP-Requests über dieselbe TCP-Verbindung** seriell verarbeitet werden müssen. Ist eine Anfrage blockiert oder langsam, können **nachfolgende Requests nicht verarbeitet werden**, obwohl sie unabhängig wären. Dadurch entstehen **unnötige Verzögerungen** beim Laden mehrerer Ressourcen.

2. Welche Nachteile von HTTP/1.1 werden mit HTTP/2.0 gelöst?

HTTP/2.0 löst zentrale Nachteile von **HTTP/1.1**, insbesondere **Head-of-Line-Blocking auf Anwendungsebene, mehrfache parallele TCP-Verbindungen, hohen Header-Overhead** und **fehlende Priorisierung**. Dies geschieht durch **Multiplexing mehrerer Streams über eine TCP-Verbindung, binäre Frames, HPACK-Header-Kompression** und **Stream-Priorisierung**.

3. Wie viele TCP-Verbindungen werden bei HTTP/2.0 für die Kommunikation mit einem Server verwendet?

Bei **HTTP/2.0** wird in der Regel **nur eine einzige TCP-Verbindung pro Server** verwendet, über die **mehrere parallele Streams** gleichzeitig abgewickelt werden. Dadurch entfällt die Notwendigkeit mehrerer paralleler TCP-Verbindungen wie bei HTTP/1.1.

4. Was ist Protocol Buffers?

Protocol Buffers (Protobuf) ist ein **binäres, plattform- und sprachunabhängiges Serialisierungsformat**, das von Google entwickelt wurde. Es dient zur **strukturierten Definition von Daten und Services** in sogenannten **.proto-Dateien** und ermöglicht eine **effiziente, kompakte und typisierte Datenübertragung**, insbesondere in Verbindung mit **gRPC**.

5. Wie unterscheidet sich eine gRPC-Schnittstelle von einer REST-basierten Schnittstelle?

Eine **gRPC-Schnittstelle** ist **RPC-basiert**, nutzt **HTTP/2** als Transportprotokoll und verwendet **Protocol Buffers** als **binäres Nachrichtenformat**, was zu **höherer Performance** und **starker Typisierung** führt. Eine **REST-basierte Schnittstelle** ist **ressourcenorientiert**, verwendet meist **HTTP/1.1** und **JSON** als textbasiertes Format und ist besonders **browserfreundlich**. gRPC eignet sich vor allem für **interne Service-zu-Service-Kommunikation**, während REST für **öffentliche Web-APIs** bevorzugt wird.

Klausur-Merksatz

HTTP/1.1 leidet unter **Head-of-Line-Blocking**, **HTTP/2** nutzt eine **TCP-Verbindung mit Multiplexing**, **Protobuf** ist ein binäres IDL-Format und **gRPC** ist eine performante, **RPC-basierte Alternative zu REST**.

1. Welche Daten sollte man cachen und welche besser nicht?

Geeignet für **Caching** sind Daten, die **häufig gelesen, selten geändert** und **nicht hochgradig kritisch** sind, zum Beispiel **statischer Web-Content, Konfigurationsdaten** oder **Berechnungsergebnisse**. Nicht geeignet sind Daten, die **sehr häufig geändert werden, starke Konsistenzanforderungen** haben oder **personenbezogen und sicherheitskritisch** sind, da veraltete Cache-Einträge zu **Inkonsistenzen** führen können.

2. Wie kann der Browser Cache kontrolliert werden?

Der **Browser Cache** wird über **HTTP-Response-Header** gesteuert, insbesondere über **Cache-Control, max-age, no-cache, no-store, public** und **private**. Zusätzlich können **ETag** und **Last-Modified** zur **Validierung gecachter Inhalte** verwendet werden. Durch **Cache Busting** (z. B. Versionsnummern oder Hashes in URLs) lassen sich aktualisierte Ressourcen erzwingen.

3. Wieso ist ein Distributed Cache schneller als eine Datenbank?

Ein **Distributed Cache** ist schneller als eine Datenbank, da er Daten **im Arbeitsspeicher (RAM)** speichert und dadurch **keine langsamen I/O-Operationen** auf Festplatten benötigt. Zudem ist er **einfach strukturiert (Key-Value-Zugriff)**, verteilt die Last auf mehrere Knoten und reduziert **Netzwerk- und Datenbankzugriffe**, was zu **geringerer Latenz** und **höherem Durchsatz** führt.

4. Worin unterscheidet sich der Key eines Redis von einem Primärschlüssel einer SQL-Datenbank?

Ein **Redis-Key** ist ein **frei definierbarer, global eindeutiger Schlüssel**, der direkt auf einen **Wert im In-Memory-Key-Value-Store** verweist und **keine relationale Bedeutung** besitzt. Ein **Primärschlüssel** in einer **SQL-Datenbank** identifiziert eine **Zeile innerhalb einer Tabelle**, ist Teil eines **Schemas** und unterliegt **Constraints** sowie **Beziehungen** zu anderen Tabellen.

5. Wofür steht die Abkürzung TTL?

TTL steht für **Time To Live** und beschreibt die **Lebensdauer eines Cache-Eintrags**. Nach Ablauf der TTL wird der Eintrag **automatisch als ungültig markiert oder gelöscht**, um **veraltete Daten** zu vermeiden.

Klausur-Merksatz

Gecacht werden häufig gelesene, selten geänderte Daten; Browser-Caches werden per HTTP-Header gesteuert; Distributed Caches sind schnell durch RAM-Zugriff; Redis nutzt freie Keys statt relationaler Primärschlüssel; TTL definiert die Lebensdauer eines Cache-Eintrags.

1. Kann der gleiche Key für unterschiedliche Datentypen verwendet werden?

Nein, in **Redis** kann ein **Key** immer nur **genau einen Datentyp** besitzen. Ein Key ist eindeutig einem Typ wie **String**, **List**, **Set**, **Hash** oder **Sorted Set** zugeordnet. Wird versucht, einen bestehenden Key mit einem **anderen Datentyp** zu verwenden, liefert Redis einen **Type Error**.

2. Kann der Datentyp nach der Erzeugung geändert werden?

Nein, der **Datentyp eines Redis-Keys kann nicht geändert werden**. Um einen anderen Datentyp zu verwenden, muss der bestehende Key **gelöscht** und anschließend **neu mit dem gewünschten Datentyp angelegt** werden.

3. Welche Datenstruktur eignet sich besonders gut zum Speichern von „Likes“ zu User Content?

Zum Speichern von **Likes** eignet sich besonders gut ein **Set** oder ein **Sorted Set**. Ein **Set** stellt sicher, dass **jeder User nur einmal liken kann** (keine Duplikate). Ein **Sorted Set** eignet sich zusätzlich, wenn Likes **gezählt oder gerankt** werden sollen, da es einen **Score** pro Element unterstützt.

4. Wann ist eine Transaktion bei einem Distributed Cache nötig?

Eine **Transaktion** ist bei einem **Distributed Cache wie Redis** notwendig, wenn **mehrere zusammengehörige Operationen atomar** ausgeführt werden müssen, um **Inkonsistenzen bei parallelem Zugriff** zu vermeiden. Typische Beispiele sind **gleichzeitiges Ändern mehrerer Keys** oder **abhängige Lese-/Schreiboperationen**, die entweder **alle erfolgreich** oder **gar nicht** ausgeführt werden sollen.

Klausur-Merksatz

Ein **Redis-Key** hat **genau einen Datentyp**, der **nicht geändert werden kann**; Likes speichert man mit **Sets oder Sorted Sets**; Transaktionen sind **nötig**, wenn **mehrere Cache-Operationen atomar** ausgeführt werden müssen.

1. Wann benötigen wir eine Transaktion?

Eine **Transaktion** wird benötigt, wenn **mehrere zusammengehörige Datenbankoperationen** als **eine logische Einheit** ausgeführt werden müssen, sodass entweder **alle Änderungen erfolgreich übernommen** oder **keine davon angewendet** werden. Sie ist insbesondere notwendig, um **Inkonsistenzen** bei **Fehlern** oder **parallelen Zugriffen** zu vermeiden.

2. Was sind die vier Schlüsseleigenschaften einer Transaktion (ACID)?

Die vier Schlüsseleigenschaften einer Transaktion sind **Atomicity, Consistency, Isolation** und **Durability**.

Atomicity garantiert, dass alle Operationen einer Transaktion vollständig oder gar nicht ausgeführt werden.

Consistency stellt sicher, dass die Datenbank nach der Transaktion alle **Constraints und Regeln** erfüllt.

Isolation sorgt dafür, dass parallele Transaktionen sich **nicht gegenseitig beeinflussen**.

Durability garantiert, dass einmal bestätigte Änderungen **dauerhaft gespeichert** bleiben, selbst bei Systemausfällen.

3. Wie wird ein Transaction Rollback getriggert?

Ein **Transaction Rollback** wird ausgelöst, wenn während einer Transaktion ein **Fehler**, eine **Exception** oder ein **Constraint-Verstoß** auftritt. In containerverwalteten Transaktionen geschieht dies typischerweise automatisch bei **RuntimeExceptions**, andernfalls kann ein Rollback **explizit programmgesteuert** ausgelöst werden.

4. Wie reduzieren wir die Angreifbarkeit einer Anwendung?

Die **Angreifbarkeit einer Anwendung** wird reduziert durch **frühe und konsequente Validierung, Eingabekontrollen an den Systemgrenzen, Zugriffsbeschränkungen, Transaktionen**, sowie durch die Einhaltung von **Security Best Practices** wie dem **Prinzip der minimalen Rechte**. Je früher fehlerhafte oder manipulierte Daten erkannt werden, desto sicherer ist die Anwendung.

5. Wie verhindern wir eine SQL Injection in JPA?

SQL Injection wird in **JPA** verhindert, indem **parametrisierte bzw. benannte Queries** verwendet werden, anstatt Benutzereingaben direkt in Query-Strings zu konkatenieren. Durch die Nutzung von **Named Parameters** übernimmt das JPA-Framework die **sichere Bindung der Parameter**, wodurch manipulierte Eingaben nicht als SQL-Code interpretiert werden können.

Klausur-Merksatz

Transaktionen sichern Konsistenz durch ACID, Rollbacks erfolgen bei Fehlern, Angriffe werden durch frühe Validierung reduziert und SQL Injection wird in JPA durch parametrisierte Queries verhindert.

1. Was sind die fünf wesentlichen Eigenschaften einer Microservice-Architektur?

Eine **Microservice-Architektur** besteht aus **kleinen, fachlich abgegrenzten Services**, die **lose gekoppelt, unabhängig deploybar, hoch wartbar und testbar** sowie **um Geschäftsdomänen (Business Capabilities)** herum organisiert sind. Jeder Microservice gehört typischerweise einem **kleinen, eigenverantwortlichen Team** und besitzt eine **klare Schnittstelle**.

2. Worin unterscheidet sich eine Single Page Application von einem Composite User Interface?

Eine **Single Page Application (SPA)** stellt die gesamte Benutzeroberfläche als **eine Client-Applikation** bereit, die über **APIs mehrere Microservices** anspricht. Ein **Composite User Interface** hingegen setzt sich aus **mehreren UI-Fragmenten** zusammen, die jeweils von **unterschiedlichen Microservices** geliefert und server- oder clientseitig **zusammengesetzt** werden.

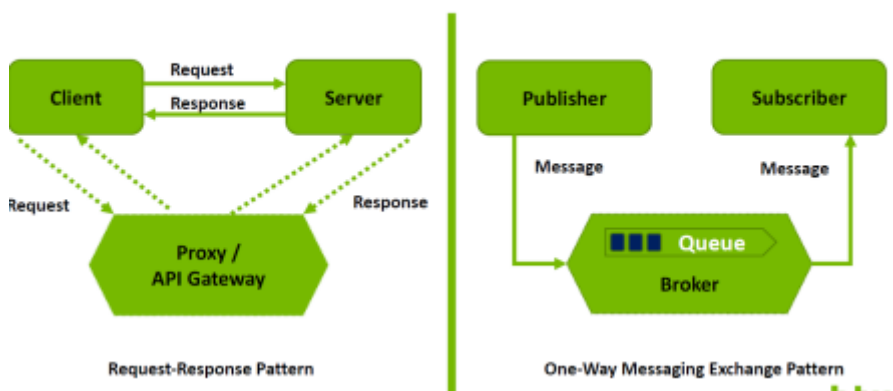
3. Mit welchen Situationen muss ein Microservice umgehen können, und wie kann das API Gateway dabei helfen?

Ein **Microservice** muss mit **Netzwerklatenzen, Time-outs, Service-Ausfällen, Lastspitzen** und **instabilen Abhängigkeiten** umgehen können. Das **API Gateway** unterstützt dabei durch **Rate Limiting, Timeouts, Retries, Circuit Breaker, Request Routing** und **Versionierung**, sodass Clients von internen Problemen abgeschirmt werden.

4. Welche zwei wesentlichen Communication Pattern gibt es?

Die zwei wesentlichen **Communication Pattern** sind das **Request-Response Pattern**, bei dem Client und Service **synchron** kommunizieren, sowie das **One-Way Messaging (Publish-Subscribe)**, bei dem Services **asynchron über Nachrichtenbroker** entkoppelt interagieren.

Communication Pattern



Klausur-Merksatz

Microservices sind lose gekoppelt, unabhängig deploybar und fachlich geschnitten; SPA ≠ Composite UI; Services müssen Fehler und Latenzen aushalten, unterstützt durch ein API Gateway; kommuniziert wird synchron (Request-Response) oder asynchron (Messaging).

1. Wie unterscheidet sich eine Pull-Architektur von einer Push-Architektur?

Bei einer **Pull-Architektur** fragt der **Client aktiv und periodisch** beim Server nach neuen Daten (Polling), unabhängig davon, ob sich etwas geändert hat. Bei einer **Push-Architektur** informiert der **Server den Client automatisch**, sobald neue Daten oder Ereignisse vorliegen. Push-Architekturen sind effizienter, da sie **unnötige Anfragen vermeiden** und **Echtzeitkommunikation** ermöglichen.

2. Welche Vorteile bietet die asynchrone Verarbeitung?

Die **asynchrone Verarbeitung** entkoppelt **Producer und Consumer**, sodass langsame oder ausgefallene Konsumenten **keinen Einfluss auf den Sender** haben. Sie verbessert die **Skalierbarkeit**, erhöht die **Fehlertoleranz** und erlaubt eine **flexible Erweiterung** des Systems, da neue Konsumenten hinzugefügt werden können, ohne bestehende Komponenten zu verändern.

3. Wann benötigt man eine Message Queue?

Eine **Message Queue** wird benötigt, wenn Aufgaben **zeitlich entkoppelt, zuverlässig verarbeitet** oder **Lastspitzen abgefangen** werden sollen. Sie ist besonders sinnvoll, wenn Prozesse **asynchron, parallel** oder **fehlertolerant** ausgeführt werden müssen, etwa bei Hintergrundjobs, Event-Verarbeitung oder Microservice-Kommunikation.

4. Aus welchen Bestandteilen besteht ein Topic in Kafka?

Ein **Kafka Topic** besteht aus mehreren **Partitionen**, die jeweils eine **geordnete Folge von Events** enthalten. Innerhalb einer Partition sind Events durch **Offsets eindeutig nummeriert** und strikt **geordnet**, während die Reihenfolge **zwischen Partitionen nicht garantiert** ist. Topics dienen als **logische Kategorie** für zusammengehörige Events.

5. Mit welchem Verfahren wird ein Event einer Partition zugeteilt?

Die Zuteilung eines Events zu einer Partition erfolgt über eine **Hash-Funktion auf dem Event-Key**. Events mit **dem gleichen Key** werden immer **derselben Partition** zugeordnet, wodurch die **Reihenfolge dieser Events garantiert** bleibt. Fehlt ein Key, erfolgt die Zuteilung meist **round-robin**.

Klausur-Merksatz (sehr wichtig)

Pull fragt aktiv ab, Push informiert automatisch; asynchrone Verarbeitung entkoppelt Systeme; Message Queues puffern und entlasten; Kafka Topics bestehen aus Partitionen mit Offsets; die Partitionierung erfolgt über Hashing des Event-Keys.